
Servidores Web de Altas Prestaciones



¿Rinde según lo esperado?
¿Podría mejorarse el rendimiento?



Tema 6: Medir prestaciones de la granja web



ICAR

INGENIERÍA DE COMPUTADORES,
AUTOMÁTICA Y ROBÓTICA



**UNIVERSIDAD
DE GRANADA**



Índice



1. Introducción

2. Conexiones por segundo

3. Número de conexiones concurrentes

4. Rendimiento, en bits por segundo

5. Tipos de tráfico

6. Límite en las prestaciones

7. ¿Cómo medir rendimiento?





Introducción

Medir las prestaciones de nuestro sistema web = comprobar si cumplen unos mínimos requisitos de rendimiento:

- los servidores finales
- los dispositivos de balanceo
- elementos de red

Aplicar una metodología de test de prestaciones para detectar posibles problemas de rendimiento.





Introducción

Principal necesidad de hacer los tests:

- No son exclusivamente las caídas o errores de programación factores que influyen en el rendimiento.
- Detectar posibles cuellos de botella e ineficiencias.
- Detectar límite del sistema.

Limitaciones de los tests:

- Dificultad para hacer pruebas en un entorno de producción.
- Dificultad para simular el comportamiento de los usuarios.





Introducción

Muy importante medir las prestaciones de los dispositivos de balanceo.

Según el sitio web, podemos contemplar diferentes métricas, entre las que destacan:

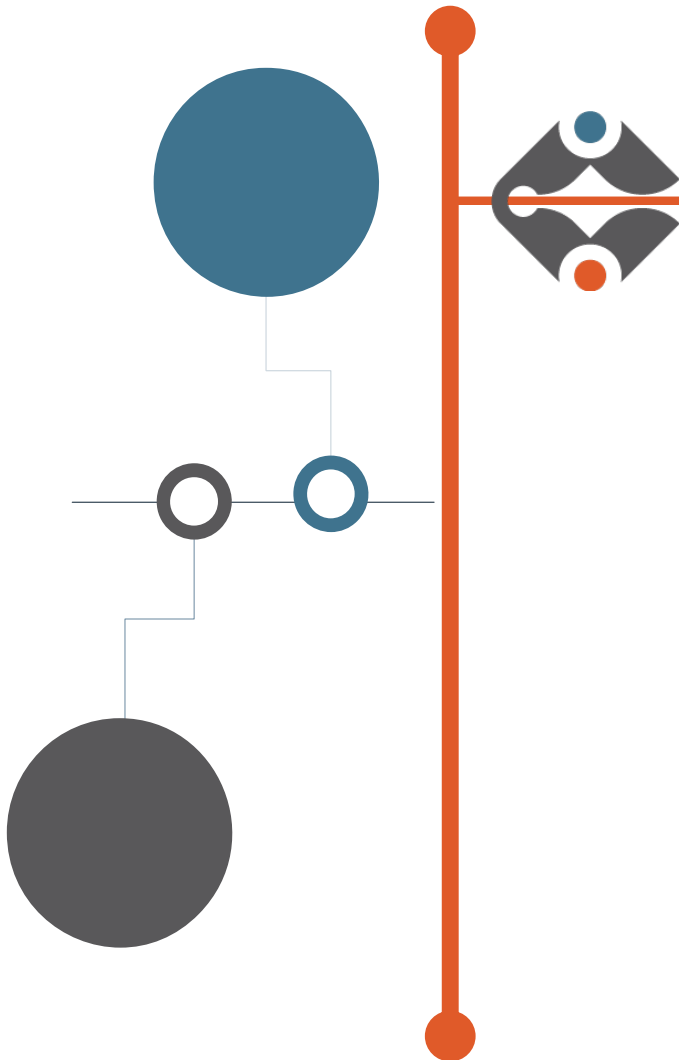
- conexiones por segundo
- número total de conexiones concurrentes
- rendimiento (en bits por segundo)





Índice

1. Introducción
2. Conexiones por segundo
3. Número de conexiones concurrentes
4. Rendimiento, en bits por segundo
5. Tipos de tráfico
6. Límite en las prestaciones
7. ¿Cómo medir rendimiento?





Conexiones por segundo

Es una de las métricas más importantes cuando hablamos del rendimiento de servidores web.

Hace referencia al número de conexiones de entrada que cierto dispositivo puede manejar por segundo.

También se llama transacciones por segundo o sesiones por segundo.





Conexiones por segundo

Es un factor determinante, ya que abrir y cerrar conexiones HTTP resulta muy costoso.

En el nivel que estamos tratando, es la operación principal.

Para enviar datos hay que llevar a cabo una serie de pasos que pueden llegar a sobrecargar el dispositivo de red.

Las aperturas y cierres de conexiones consumen muchos recursos.



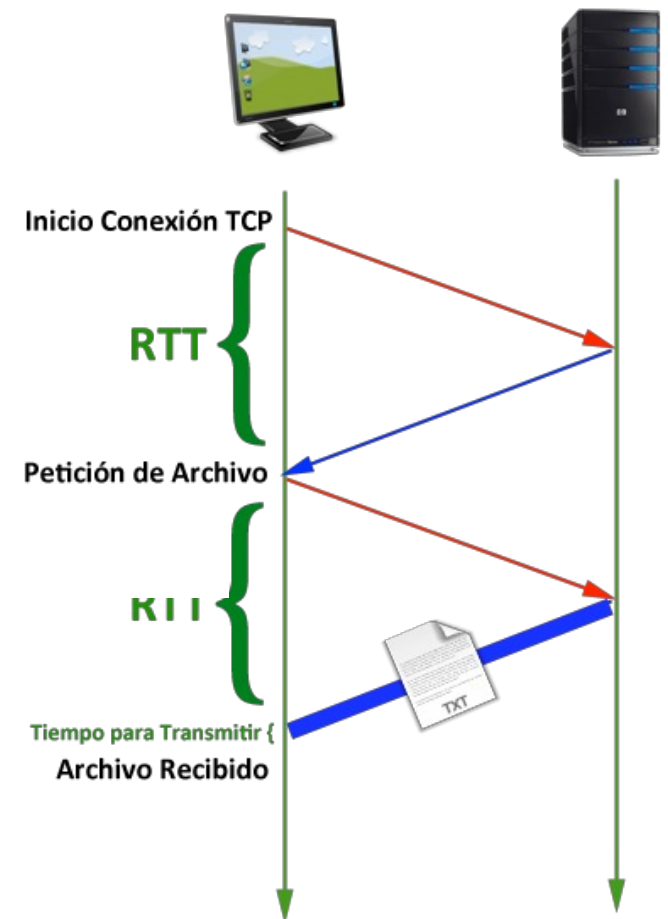


Conexiones por segundo

Pasos para establecer una conexión HTTP:

- el cliente inicia la conexión HTTP enviando un paquete TCP SYN al puerto 80 del servidor web
- el servidor web envía un paquete ACK al cliente seguido de otro SYN
- el cliente envía un paquete ACK como respuesta

Ahora ya pueden comenzar a enviarse datos desde el servidor al cliente (normalmente será una página web).





Conexiones por segundo

La velocidad a la que se gestionan las aperturas y cierres de conexiones es fundamental.

Si cierto servidor web tiene un tráfico HTTP alto (muchas peticiones), conexiones por segundo podrá ser la métrica más interesante.





Conexiones por segundo

Ver el número de conexiones por segundo.

```
netstat -an | grep :80 | sort
```

```
netstat | grep http | wc -l
```

```
netstat -n -p | grep SYN_REC | sort -u
```

Habilitar stats en Nginx y ver conexiones activas y conexiones por segundo:

```
location /
{
    proxy_pass http://balanceo_jmsoto;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_http_version 1.1;
    proxy_set_header Connection "";
}

location /nginx_status{
    stub_status;
    allow 192.168.56.1;
    deny all;
}
```



No es seguro | 192.168.56.103/nginx_status

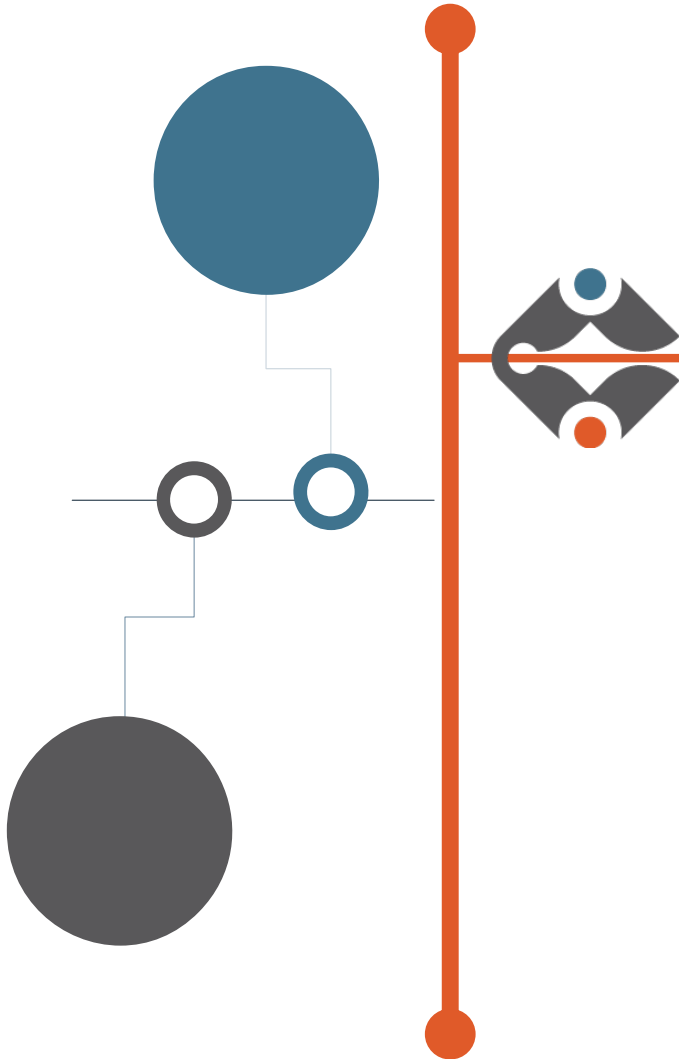
```
Active connections: 12
server accepts handled requests
2337 2337 2362
Reading: 0 Writing: 10 Waiting: 2
```





Índice

1. Introducción
2. Conexiones por segundo
3. Número de conexiones concurrentes
4. Rendimiento, en bits por segundo
5. Tipos de tráfico
6. Límite en las prestaciones
7. ¿Cómo medir rendimiento?





Número de conexiones concurrentes

Métrica para determinar cuántas sesiones de usuario TCP puede manejar el balanceador al mismo tiempo.

El número de conexiones que están activas y abiertas en un momento dado en el servidor o granja web. No se trata solo de conexiones establecidas, sino de aquellas que están activamente en uso, transmitiendo datos o esperando respuesta.

Limitado por la memoria o el procesador del dispositivo.

Varía desde varios miles hasta millones.
Límite teórico (realmente no es tan alta).





Número de conexiones concurrentes

Esto en cuanto a las conexiones TCP...

Sin embargo, para el tráfico UDP (streaming o tráfico DNS) el número de conexiones concurrentes no es un factor que afecte, ya que se trata de un protocolo "sin conexión":

- el receptor no reconoce haber recibido paquetes.

No hay una fase de establecimiento de la conexión.

No se mantiene información de estado.

TCP mantiene información sobre el estado de la conexión para garantizar un servicio fiable de transferencia de datos y control de congestión.





Número de conexiones concurrentes

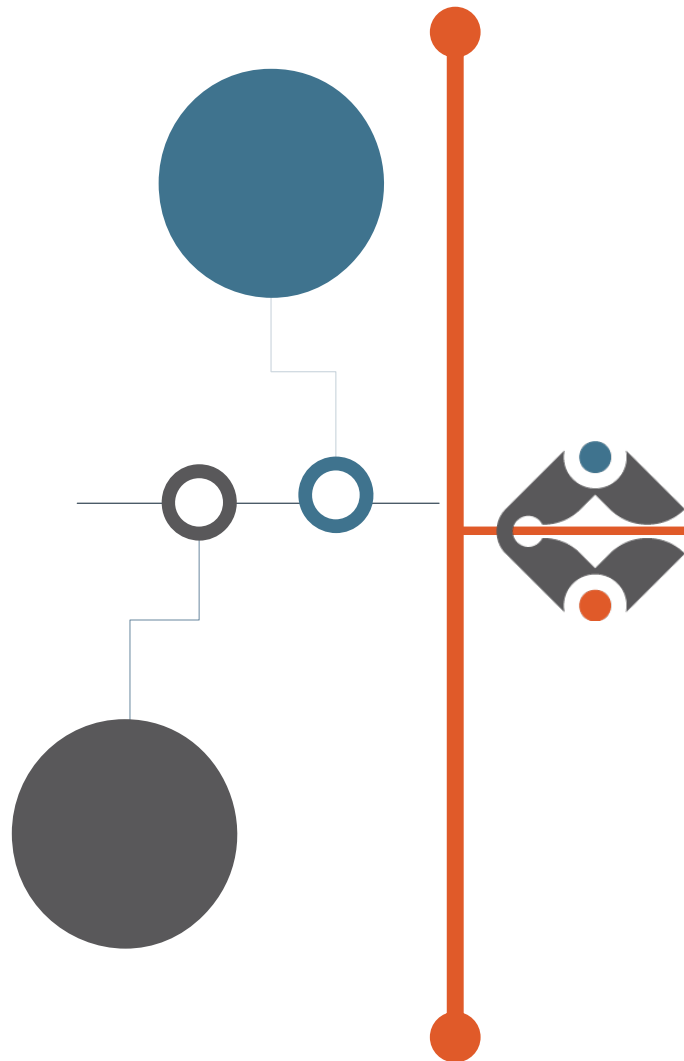
Hoy en día, las webs de vídeos como Youtube o Vimeo, utilizan TCP ya que algunas organizaciones bloquean el tráfico UDP por cuestiones de seguridad.

También se usa TCP para no colapsar el servidor ya que TCP provee control de congestión.





Índice



1. Introducción
2. Conexiones por segundo
3. Número de conexiones concurrentes
4. Rendimiento, en bits por segundo
5. Tipos de tráfico
6. Límite en las prestaciones
7. ¿Cómo medir rendimiento?





Rendimiento, en bits por segundo

Hace referencia a la velocidad a la que el balanceador maneja y pasa el tráfico.

Todos los dispositivos tienen una serie de factores que acaban limitando las prestaciones, basados en la estructura interna (hardware y software).

Algunos desarrolladores de balanceadores de carga sólo soportan Fast Ethernet, limitándolos así a 100Mbps.

Algunas implementaciones no tienen el hardware o el software adecuado, con lo que quedan limitados a transferencias máximas de 80Mbps.





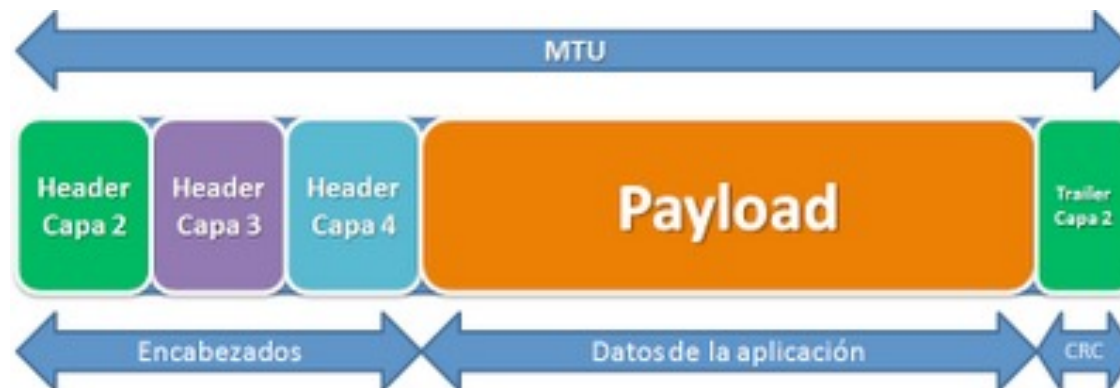
Rendimiento, en bits por segundo

Se mide en bits por segundo. Es combinación de las variables "tamaño del paquete" y "paquetes por segundo".

Unidad Máxima de Transmisión (MTU, por sus siglas en inglés)

- El tamaño está definido por la red específica y varía según el tipo de red. Por ejemplo, Red Ethernet, el MTU es de **1.5KB**

Si hay que enviar más datos, se trocean en paquetes de este tamaño máximo.





Rendimiento, en bits por segundo

Un MTU más grande permite que se envíen más datos en un solo paquete (menos overhead). Sin embargo, puede llevar a una fragmentación porque haya paquetes de menos tamaño que el MTU.

La elección de una **MTU óptima** es un **equilibrio** entre evitar la **fragmentación** y minimizar el **overhead** de transmisión.

- Uso de herramientas de diagnóstico: Se pueden utilizar herramientas como ping o traceroute con opciones de "no fragmentar" para determinar la MTU máxima que puede pasar sin fragmentación a través de una ruta de red específica. Este proceso implica enviar paquetes de diferentes tamaños para ver cuál es el máximo que pasa sin fragmentarse.

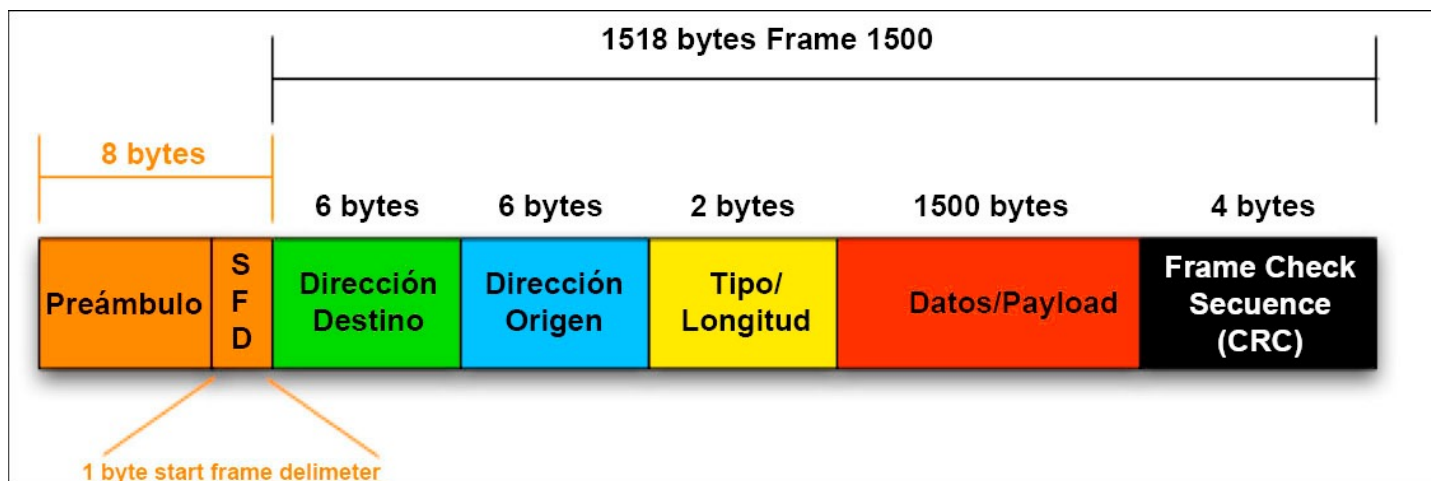




Rendimiento, en bits por segundo

Ejemplo:

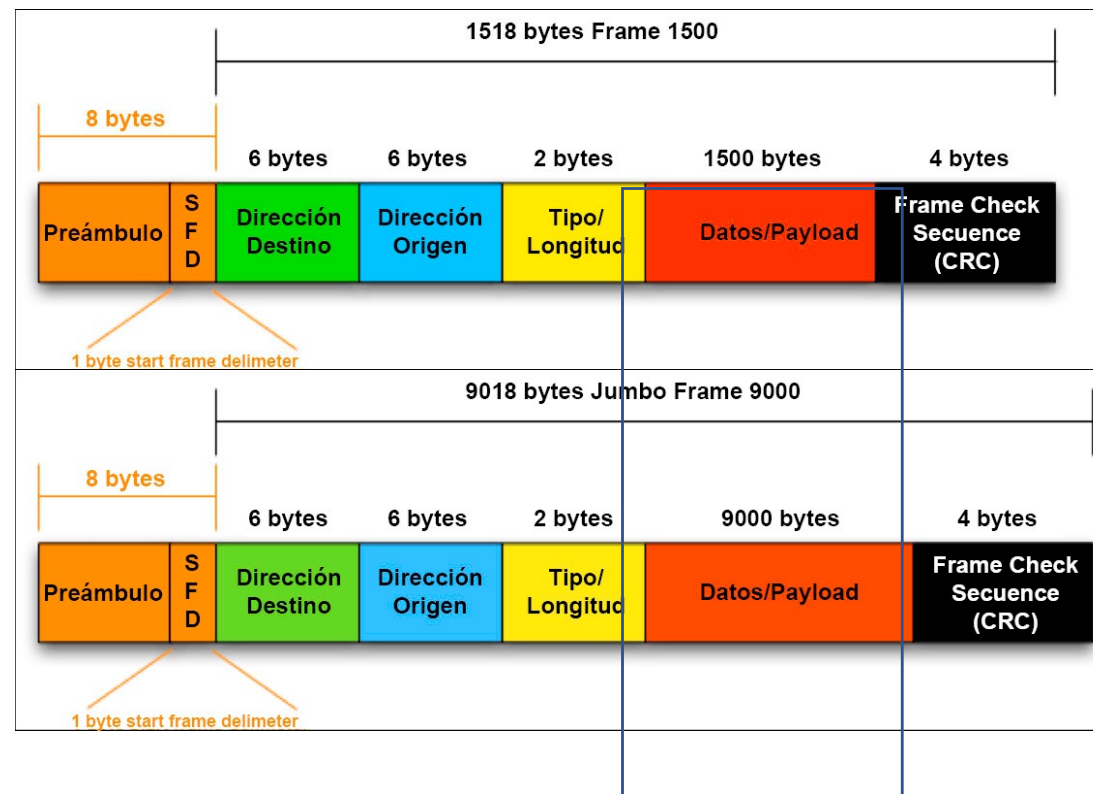
- un acceso por HTTP usando el método GET a un recurso de 100 bytes podrá servirse en un solo paquete.
- un acceso por GET a un archivo de 32KB necesitará 21 paquetes, con 1.5KB de información útil (payload) en cada uno.





¿Y para grandes transferencias?

¡Jumbo Frame! “con la salida de las redes Gigabit se implementó la posibilidad de modificar el tamaño de esos paquetes para optimizar tanto tráfico como transferencia de información”





¿Y para grandes transferencias?

¡Jumbo Frame! - Ventajas

Al ser paquetes más grandes requieren de menor dedicación de CPU para su procesamiento en tarjetas de red, routers, etc, además de llegar antes

Se minimiza el uso de bytes para la asignación de los datos que acompañan a la MTU, dedicando más bytes a datos enviados.

Aliviamos la carga de protocolo y menor fragmentación de la información por la red.

Se mejora la carga en los programas que gestionan el tráfico de red, como son los Firewalls (Cortafuegos).





¿Y para grandes transferencias?

¡Jumbo Frame! - Desventajas

Al aumentar el tamaño de los paquetes aumenta la latencia puesto que el tiempo de envío de los mismos es mayor, no siendo conveniente en redes con fuerte streaming.

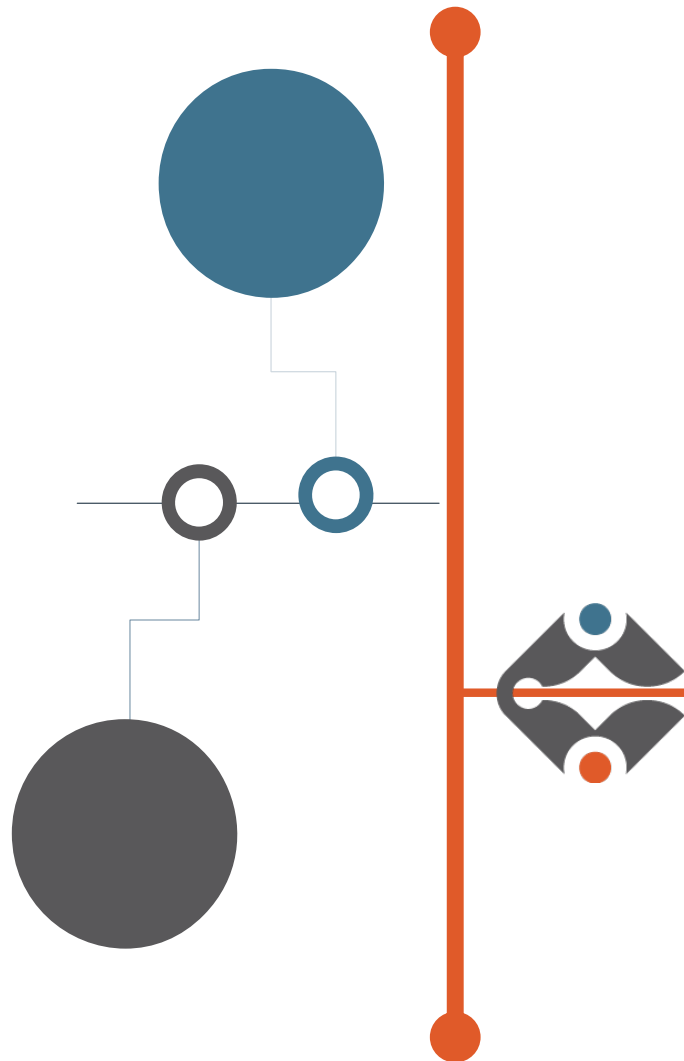
En redes de baja/media calidad provoca que si un paquete no pasa la comprobación de errores debe volverse a reenviar, siendo mayor la información que se reenvía y el tiempo que se pierde en la tarea.

No es muy recomendable forzar el uso de Jumbo Frames (paquetes de información grandes) con redes que manejan paquetes pequeños (P2P) puesto que no se rinde al 100% de la configuración, es conveniente probar la administración por parte del router de esta mezcla.





Índice



1. Introducción
2. Conexiones por segundo
3. Número de conexiones concurrentes
4. Rendimiento, en bits por segundo
5. Tipos de tráfico
6. Límite en las prestaciones
7. ¿Cómo medir rendimiento?





Tipos de tráfico

Hay patrones de tráfico muy comunes en sitios web:

- HTTP
- FTP o streaming
- tienda web

Patrón de tráfico	Métrica más importante	Segunda métrica más importante	Métrica menos importante
HTTP	Conexiones por segundo	Rendimiento	Total de conexiones concurrentes
FTP/streaming	Rendimiento	Total de conexiones concurrentes	Conexiones por segundo
Tienda web	Total de conexiones concurrentes	Conexiones por segundo	Rendimiento





Tipos de tráfico

1. Tráfico HTTP: Consume ancho de banda intensivamente y genera muchas conexiones por segundo.

HTTP 1.0

- **Versión inicial:** Introducido en 1996, HTTP 1.0 es la primera versión formalmente definida del protocolo.
- **Conexiones no persistentes:** Cada solicitud de un archivo requería una conexión TCP separada. Esto significaba que para cargar una página web completa con múltiples imágenes, CSS y archivos de JavaScript, el navegador tenía que establecer una conexión TCP por cada uno de estos recursos: 1 objeto por conexión.
- **Limitaciones:** La necesidad de múltiples conexiones hacía que HTTP 1.0 fuera ineficiente en términos de tiempo y recursos, especialmente para páginas web complejas.
- **Cabeceras simples:** HTTP 1.0 utiliza un conjunto básico de cabeceras que permite la transmisión de metadatos sobre el navegador y el servidor, pero con opciones limitadas de negociación y control.





Tipos de tráfico

1. Tráfico HTTP: Consume ancho de banda intensivamente y genera muchas conexiones por segundo.

HTTP 1.1

- **Introducido en 1997:** HTTP 1.1 fue diseñado para superar muchas de las limitaciones de la versión 1.0.
- **Conexiones persistentes:** HTTP 1.1 introdujo el concepto de conexiones persistentes (también conocidas como conexiones "keep-alive"), lo que permite múltiples solicitudes y respuestas en una sola conexión TCP. Esto reduce la sobrecarga de establecimiento de conexión y mejora significativamente la eficiencia: Varios objetos por conexión.
- **Caché y gestión de errores mejoradas:** HTTP 1.1 ofrece controles más sofisticados sobre la caché y mejores protocolos de gestión de errores.
- **Transferencia fragmentada:** Esta característica permite que los servidores envíen parte de una respuesta cuando está lista, lo que es útil para contenido grande y dinámico.
- **Host virtual:** HTTP 1.1 permite la utilización de múltiples dominios desde una única dirección IP, una capacidad crítica para el hosting web moderno.





Tipos de tráfico

1. Tráfico HTTP: Consume ancho de banda intensivamente y genera muchas conexiones por segundo.

HTTP/2

- **Aprobado en 2015:** HTTP/2 es la evolución de HTTP 1.1, diseñado para mejorar aún más la eficiencia de la comunicación en Internet.
- **Multiplexación:** HTTP/2 puede enviar múltiples solicitudes y respuestas en paralelo a través de una única conexión TCP, superando las limitaciones de HTTP 1.1 donde las solicitudes se procesan en orden (conocido como "blocking").
- **Compresión de cabeceras:** Reduce la sobrecarga de datos mediante la compresión de cabeceras entre conexiones.
- **Priorización de solicitudes:** Permite que los clientes prioricen las solicitudes, ayudando a los servidores a enviar recursos críticos primero.





Tipos de tráfico

2. Tráfico FTP / streaming:

Tras una conexión inicial (ya que usa UDP como protocolo), se envía una gran cantidad de información.

El número de conexiones para este tipo de tráfico es muy bajo comparado con la cantidad de información enviada.

Consumen el ancho de banda máximo rápidamente.





3. Tráfico tipo “tienda web”:

La velocidad es el factor más importante.

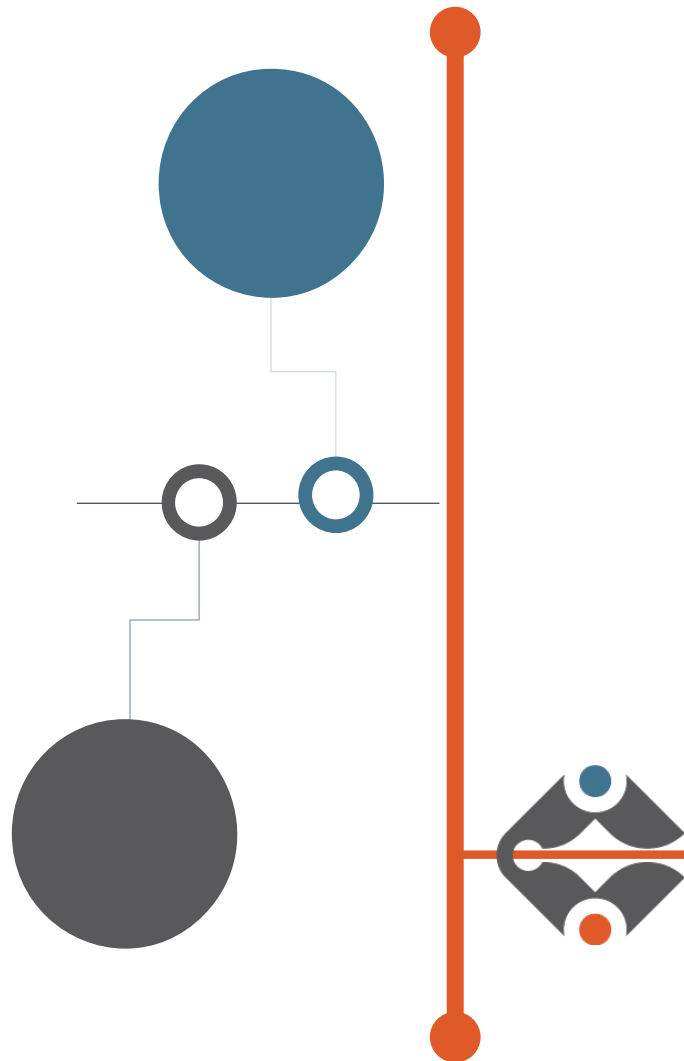
Buena experiencia de usuario: si el usuario se desespera navegando en la tienda web, gastará poco dinero...

No se necesita un alto ancho de banda, ni tampoco va a haber demasiadas conexiones por segundo.

Sin embargo, el sitio debe dar soporte al máximo de usuarios navegando en sesiones largas al mismo tiempo.



Índice



1. Introducción
2. Conexiones por segundo
3. Número de conexiones concurrentes
4. Rendimiento, en bits por segundo
5. Tipos de tráfico
6. Límite en las prestaciones
7. ¿Cómo medir rendimiento?





Límite de las prestaciones

Existe un límite de tráfico de red suficientemente alto que produce una degradación grave en las prestaciones.

En unos casos ese límite es más fácil de alcanzar que en otros.

Llegado a ese límite los tiempos de respuesta en las conexiones HTTP se degradan completamente, haciendo imposible la conexión → perjudicando la disponibilidad



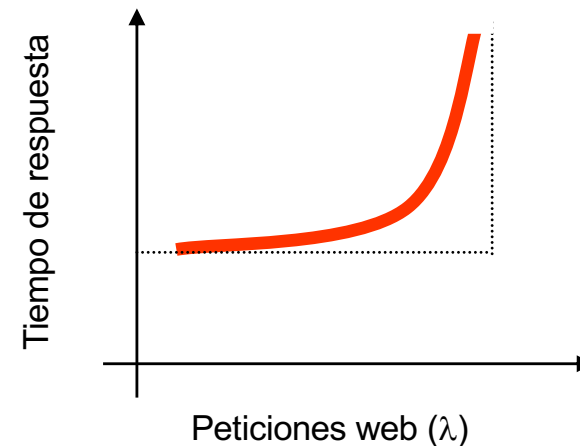


Límite de las prestaciones

Si lo representamos gráficamente:

Esta degradación en las prestaciones se debe a los cuellos de botella.

Hay que estudiar los datos de la monitorización durante las pruebas para determinar las carencias.

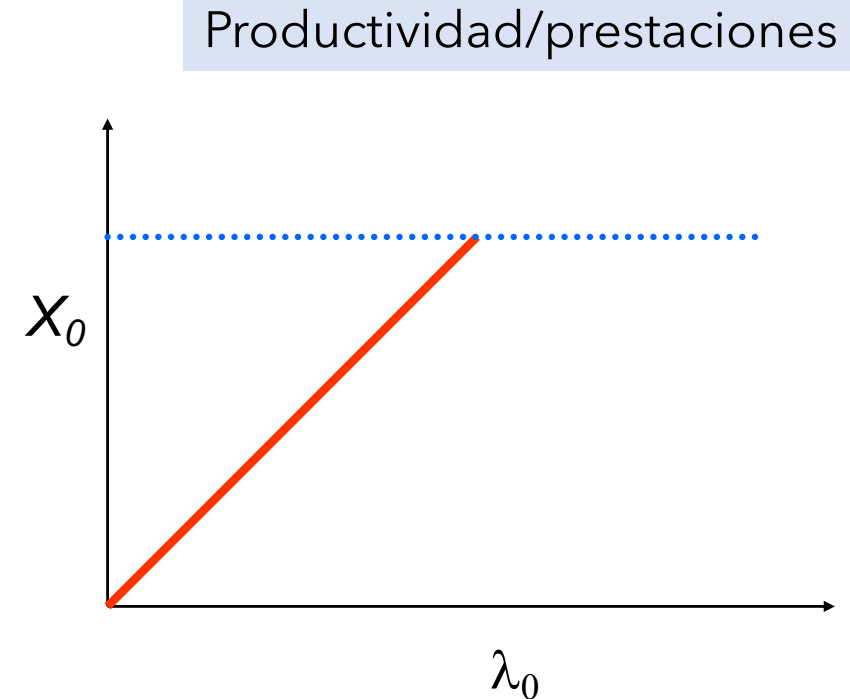
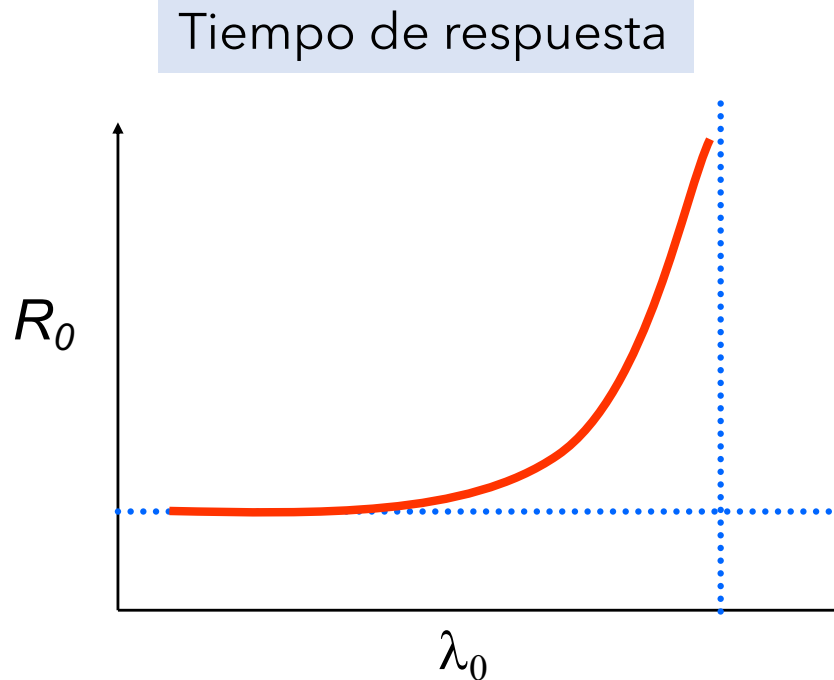




Límite de las prestaciones

Ejemplo 1: curva característica

Al subir la carga, las prestaciones/productividad se degradan y dejan de crecer.





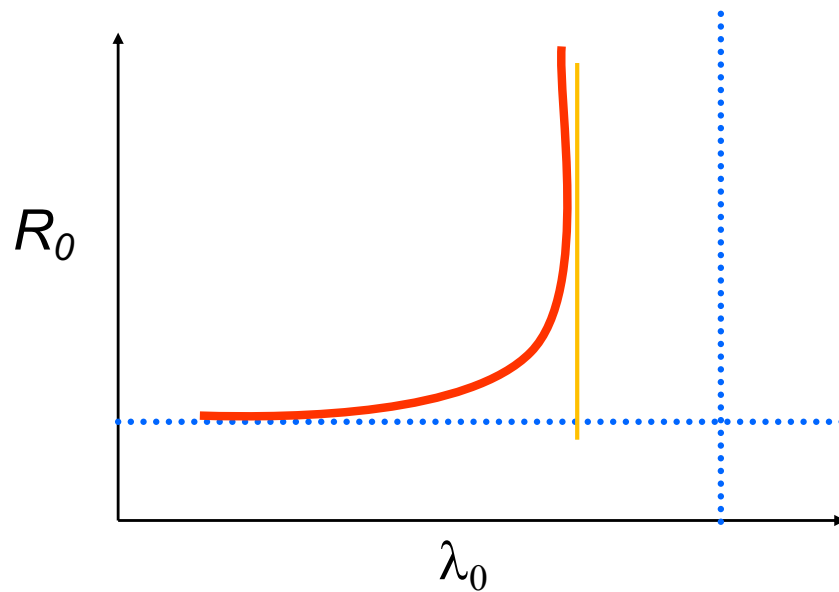
Límite de las prestaciones

Ejemplo 2: cuando ocurre algún problema...

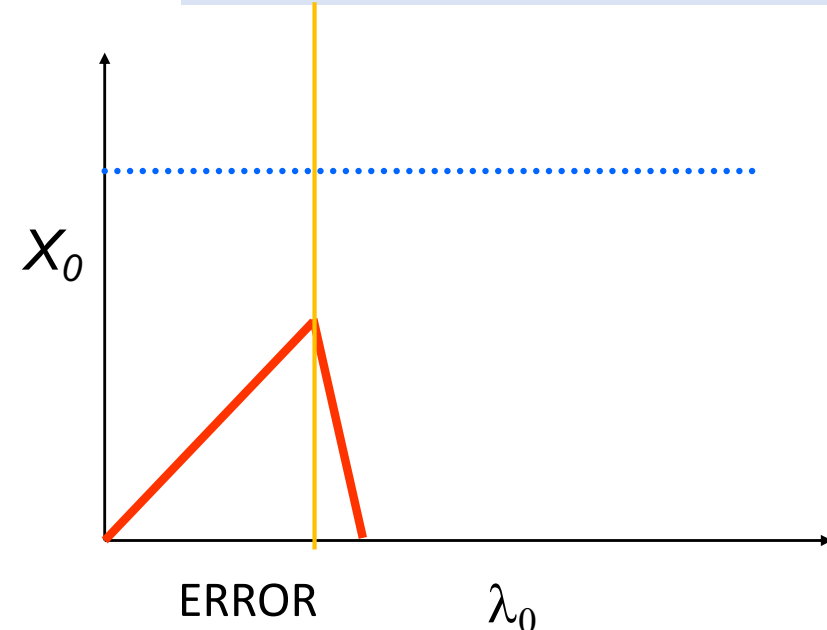
Al fallar algún servicio, caen las prestaciones → También los tiempos de respuesta, al terminar las transacciones rápidamente con un error.

En estos casos es interesante examinar los logs (acceso y error).

Tiempo de respuesta



Productividad/prestaciones





Límite de las prestaciones

Cada dispositivo de red puede comportarse de forma diferente, llegando a cuelgues o reinicios.

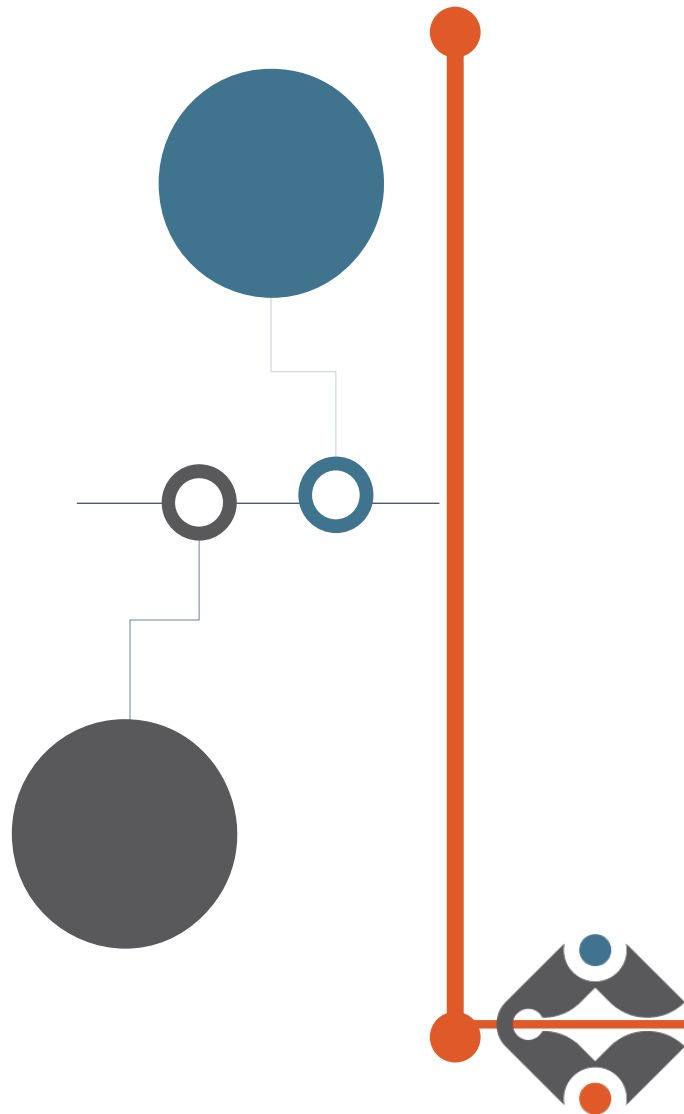
Estos límites son difíciles de alcanzar... pero a mayor número de características activas en un balanceador, será más fácil de alcanzar el límite:

- Por ejemplo: si es capaz de procesar tráfico a 90Mbps (suponemos máximo 100Mbps), puede ver reducido su rendimiento a la mitad si le pedimos que haga análisis de URLs y que de soporte de cookies (requieren uso más intensivo de la CPU para inspeccionar los paquetes completos y no solo la cabecera).





Índice



1. Introducción
2. Conexiones por segundo
3. Número de conexiones concurrentes
4. Rendimiento, en bits por segundo
5. Tipos de tráfico
6. Límite en las prestaciones
7. ¿Cómo medir rendimiento?





¿Cómo medir el rendimiento? – Análisis de tráfico

Analizar el tráfico de un sitio web es fundamental para entender cómo los usuarios interactúan con el sitio, identificar problemas de rendimiento y asegurar la seguridad de las comunicaciones en línea.

Por ejemplo:

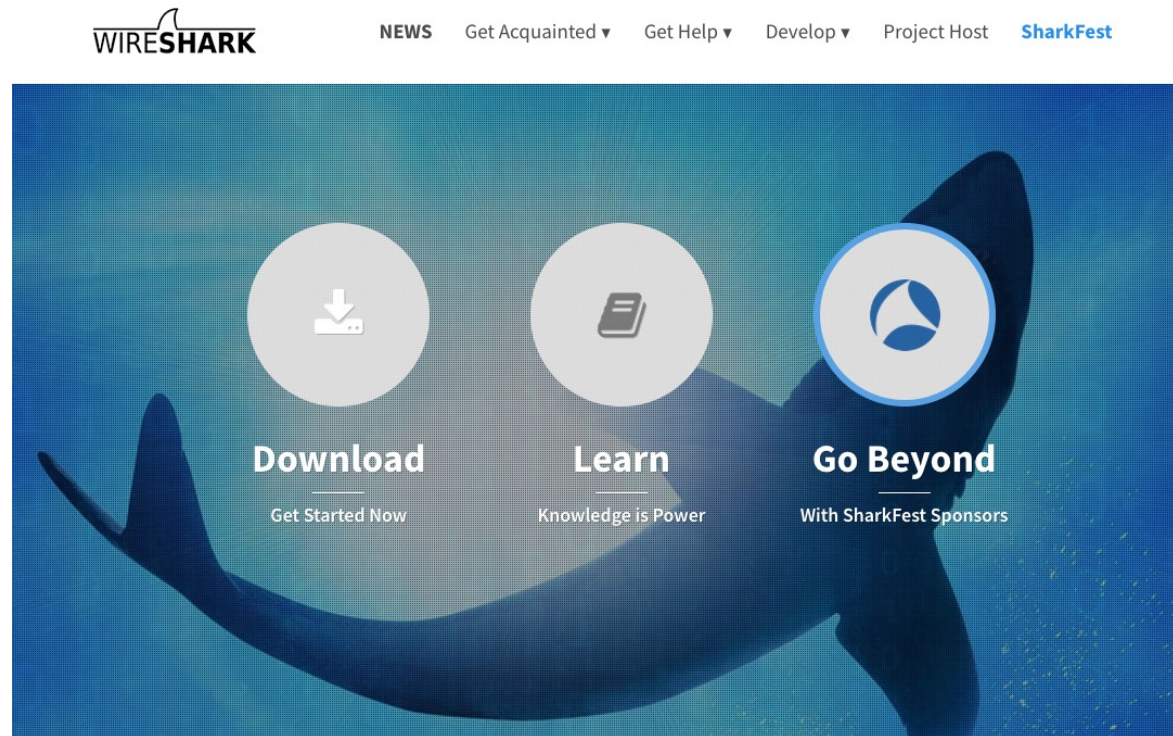
- Puedes ver cómo navegan los usuarios en tu sitio, qué páginas visitan más a menudo y dónde pueden estar experimentando problemas.
- Puedes identificar cuellos de botella en la carga de la página y otros problemas de rendimiento.
- Puedes detectar patrones sospechosos que podrían indicar un ataque, como un aumento en el tráfico no habitual, solicitudes de ubicaciones inusuales o una gran cantidad de solicitudes fallidas que podrían ser un intento de intrusión





Análisis del tráfico con Wireshark

<https://www.wireshark.org/>



Es un analizador de protocolos utilizado para realizar análisis y solucionar problemas en redes de comunicaciones, para análisis de datos y protocolos



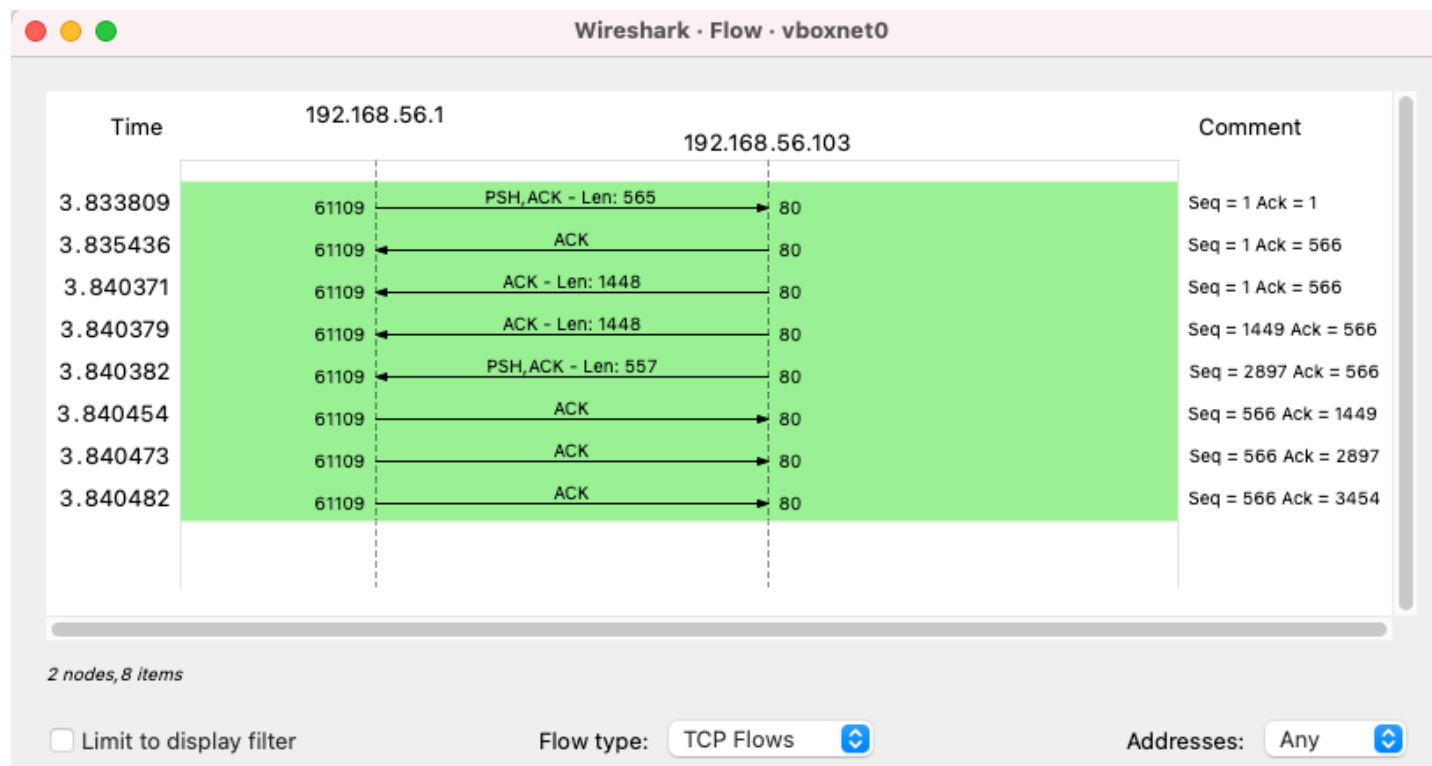


Análisis del tráfico con Wireshark

<https://www.wireshark.org/>



Establecimiento de conexión (handshake de tres vías)





Análisis del tráfico con Wireshark

Ej: Establecimiento de conexión (handshake de tres vías)

Source Port: 61676
Destination Port: 80
[Stream index: 0]
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 960932927
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 0
Acknowledgment Number (raw): 0
1011 = Header Length: 44 bytes (11)
Flags: 0x0c2 (SYN, ECN, CWR)
000. = Reserved: Not set
...0 = Nonce: Not set
.... 1... = Congestion Window Reduced (CWR): Set
.... 1.. = ECN-Echo: Set
.... ..0. = Urgent: Not set
.... ..0. = Acknowledgment: Not set
.... ..0... = Push: Not set
.... ..0.. = Reset: Not set
>1. = Syn: Set
0 = Fin: Not set
[TCP Flags: ...CE...S.]





Análisis del tráfico con Wireshark

Ej: Establecimiento de conexión (handshake de tres vías)

The screenshot displays the Wireshark interface with a packet capture of a three-way handshake. The packet list shows a TCP SYN packet from 192.168.56.1 to 192.168.56.103. The packet details pane shows the TCP header with flags SYN, ACK, and ECN. The packet bytes pane shows the raw data.

Source Port: 80
Destination Port: 61676
[Stream index: 0]
[TCP Segment Len: 0]
Sequence Number: 0 (relative sequence number)
Sequence Number (raw): 286750133
[Next Sequence Number: 1 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 960932928
1010 = Header Length: 40 bytes (10)
Flags: 0x052 (SYN, ACK, ECN)
000. = Reserved: Not set
...0 = Nonce: Not set
...0 = Congestion Window Reduced (CWR): Not set
...1. = ECN-Echo: Set
...0. = Urgent: Not set
...1. = Acknowledgment: Set
...0. = Push: Not set
...0. = Reset: Not set
...1. = Syn: Set
...0. = Fin: Not set
[TCP Flags:E.A.S.]
Window: 65160





Análisis del tráfico con Wireshark

Ej: Establecimiento de conexión (handshake de tres vías)

Source Port: 61676
Destination Port: 80
[Stream index: 0]
[TCP Segment Len: 565]
Sequence Number: 1 (relative sequence number)
Sequence Number (raw): 960932928
[Next Sequence Number: 566 (relative sequence number)]
Acknowledgment Number: 1 (relative ack number)
Acknowledgment number (raw): 286750134
1000 ... = Header Length: 32 bytes (8)
Flags: 0x018 (PSH, ACK)
000. = Reserved: Not set
...0 = Nonce: Not set
...0 = Congestion Window Reduced (CWR): Not set
...0 = ECN-Echo: Not set
...0 = Urgent: Not set
...1 = Acknowledgment: Set
...1 = Push: Set
...0 = Reset: Not set
...0 = Syn: Not set
...0 = Fin: Not set
[TCP Flags: ...]
Window: 2058

0020 38 67 f0 ec 00 50 39 46 ac 40 11 17 75 b6 80 18 8g...P9F @..u..
0030 08 0a f1 05 00 00 01 01 08 0a 31 01 33 42 7d b31.3B}<
0040 70 a7 47 45 54 20 2f 20 48 54 54 50 2f 31 2e 31 p.GET / HTTP/1.1
0050 0d 0a 48 6f 73 74 3a 20 31 39 32 2e 31 36 38 2e ..Host: 192.168.
0060 35 36 2e 31 30 33 0d 0a 43 6f 6e 6e 65 63 74 69 56.103..Connecti
0070 6f 6e 3a 20 6b 65 65 70 2d 61 6c 69 76 65 0d 0a on: keep-alive..





¿Cómo medir el rendimiento? – Tipos de pruebas

Diferentes tipos de pruebas:

Humo (Smoke): pruebas preliminares para comprobar que el sistema está listo para los siguientes tests.

Carga (Load): cargas lo más parecidas a la real. Se ejecutan en periodos cortos (1h). Para determinar los tiempos de respuesta que tendrán los usuarios.

Capacidad (Capacity): actividad creciente hasta detectar el punto de saturación.





¿Cómo medir el rendimiento? – Tipos de pruebas

Tipos de pruebas (II):

Estrés (Stress): para analizar el efecto de aplicar de forma continuada una carga por encima de la capacidad del sistema.

Sobrecarga (Overload): aplicar fuertes picos de carga durante cortos periodos.

Estabilidad (Stability): cargas lo más similares posibles a la real, aplicadas durante 1 día o 1 semana.





Durante los tests, monitorización

Durante la sesión de pruebas, recoger mediciones que nos indiquen lo que está ocurriendo en el sistema en cada momento y cómo reacciona éste en función de la carga introducida:

- Medidas de la calidad de servicio ofrecida por el sistema a los usuarios (estadísticas proporcionadas por la misma herramienta de simulación de carga)
- Medidas relativas al consumo de recursos del sistema (utilizando las herramientas del sistema operativo)





Software para los tests

Diversas herramientas para comprobar el rendimiento de servidores web. Línea de comandos y de interfaz gráfica:

- Apache Benchmark
- Siege
- Weighttp
- httpperf
- OpenWebLoad
- The Grinder
- OpenSTA
- JMeter
- Webstone (Mindcraft)
- Molotov
- Locust
- Etc. etc.

Las de línea de comandos sobrecargan menos las máquinas.





Software para los tests

Estas herramientas permiten comprobar el rendimiento de cualquier servidor web (Apache, MS Internet Information Services -IIS-, nginx, Cherokee, Tomcat, lighttpd, thttpd, etc).

Comprobar el rendimiento del hardware, software o de alguna modificación que le hayamos hecho.

Interesante combinar con WireShark!



Servidores Web de Altas Prestaciones



¿Rinde según lo esperado?
¿Podría mejorarse el rendimiento?



Tema 6: Medir prestaciones de la granja web



ICAR

INGENIERÍA DE COMPUTADORES,
AUTOMÁTICA Y ROBÓTICA



**UNIVERSIDAD
DE GRANADA**
